(Term Project – Data Science)

AI vs Human Text Detection

By

Jawwad Ahmad

F25NDOCS3W04013

Session: Fall 2025 – 2027

Supervisor

Sir. Abdul Rehman

Master of Science in Computer Science

The Islamia University of Bahawalpur
Bahawalnagar Campus

Table of Contents

1. Introduction 1

Background: 1

Problem Statement: 1

Motivation of the Study: 1

Objectives of the Project: 1

2. Dataset Description 2

Dataset Source 2

Dataset Summary 2

Dataset Features Explanation 2

3. Methodology & Implementation 3

3.1 Dataset Loading & Preparation 4

3.1.1 Import Libraries 4

3.1.2 Dataset Uploaded 4

3.1.3 Dataset Basic Exploration 4

3.2 Exploratory Data Analysis (EDA) 6

3.2.1 Content type distribution by Label 7

3.2.2 Target Variable Distribution 8

3.2.3 Univariate Analysis 9

3.2.4 Bivariate Analysis (Feature vs Target) 10

3.2.5 Multivariate Analysis (Feature Relationships) 11

3.3 Data Cleaning & Preprocessing 12

3.3.1 Check Missing Values 12

3.3.2 Handling Missing Values 13

3.3.3 Checking Duplicate Values 14

3.3.4 Remove Unnecessary Columns 14

3.3.5 Encode Categorical Features (using Onehot Encoding) 15

3.4 Feature Engineering 16

3.4.1 Separate Features and Target Variables 16

3.4.2 Train Test Split 16

3.4.3 Feature Scaling (using Standard Scaler) 16

3.5 Model Building 18

3.6 Model Evaluation 19

3.6.1 Generate Predictions & Metrics 19

3.7 Model Evaluation 20

3.7.1 Generate Predictions & Metrics _____ **Error! Bookmark not defined.**
3.7.2 Display Confusion Matrix (for best Accuracy Model) _____ 20
4. Conclusion _____ 22
4.1 Conclusion_____ 22
4.2 What I Learned?_____ 22

Chapter 1

Introduction

1.Introduction

Background:

Advances in natural language generation have enabled **artificial intelligence models** to produce text that closely resembles **human writing** in terms of **structure, grammar, and readability.** This progress has significantly reduced the gap between AI-generated and human-written content, posing new challenges for automated text classification.

Problem Statement:

This project addresses a **supervised binary classification problem** aimed at distinguishing AI-generated text from human-written text using handcrafted **linguistic and readability-based features.** The primary challenge arises from the substantial overlap in feature distributions between the two classes.

Motivation of the Study:

The motivation for this study lies in the growing demand for reliable AI-generated text detection in educational, publishing, and digital content moderation systems. Understanding the limitations of traditional machine learning approaches in this context is essential for developing more robust detection methods.

Objectives of the Project:

The objective of this project is to implement a complete data science workflow, including **exploratory data analysis, preprocessing, model training, and evaluation to** assess the effectiveness of classical machine learning models for AI vs Human text classification.

Chapter 2

Dataset Description

2.Dataset Description

Dataset Source

https://www.kaggle.com/datasets/pratyushpuri/ai-vs-human-content-detection-1000-record-in-2025/data

Dataset Summary

Total Records (Rows)	1367 text_content
Total Features (Columns)	17 columns
Target Variable	label (1 = AI-generated, 0 = Human-written)
Dataset Year:	2025 specifications

Dataset Features Explanation

Column Name	*Description*
text_content	The actual textual content of the document used for analysis and feature extraction.
content_type	Category of the text such as essay, academic article, blog, or news content.
word_count	Total number of words present in the text.
character_count	Total number of characters in the text including spaces.
sentence_count	Total number of sentences in the text.
lexical_diversity	Ratio of unique words to total words, indicating vocabulary richness.
avg_sentence_length	Average number of words per sentence in the text.
avg_word_length	Average number of characters per word.
punctuation_ratio	Proportion of punctuation marks relative to total characters.
flesch_reading_ease	Readability score indicating how easy the text is to read (higher = easier).
gunning_fog_index	Measure of text complexity and years of education needed to understand it.
grammar_errors	Number of grammatical mistakes detected in the text.
passive_voice_ratio	Percentage of sentences written in passive voice.
predictability_score	Measure of how predictable or structured the text is.
burstiness	Degree of variation in sentence length throughout the text.
sentiment_score	Numerical value representing the emotional tone of the text.
label	Target variable indicating whether the text is **AI-generated** or **Human-written**.

Chapter 3

Methodology & Implementation
(Data Science Workflow)

"This chapter describes the methodology and implementation steps followed to conduct experiments for AI vs Human text classification using machine learning techniques."

3. Methodology & Implementation

This study adopts a structured data science methodology to classify text as **AI-generated** or **human-written** using supervised machine learning techniques. The workflow begins with dataset loading and initial inspection, followed by exploratory data analysis to examine class distribution, feature behavior, and relationships among variables. Based on these insights, preprocessing steps such as handling missing values and feature scaling are applied to prepare the data for modeling.

The processed dataset is then divided into training and testing sets to enable fair evaluation. Multiple machine learning models, including probabilistic, distance-based, and ensemble approaches, are trained and evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. All experiments are conducted in the **Google Colab** environment using Python-based data science libraries, ensuring consistency and reproducibility.

Google Colab Link:

https://colab.research.google.com/drive/1IUs5znPTF9FJXhYpcvXywyL4xqjN95fn?usp=sharing

3.1 Dataset Loading & Preparation

Following steps are performed in this part:

1. Import Necessary Libraries
2. Dataset uploaded in csv format
3. Basic Exploration

3.1.1 Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

3.1.2 Dataset Uploaded

```
df = pd.read_csv(r'/content/ai_human_content_detection_dataset.csv')
```

3.1.3 Dataset Basic Exploration

- **df.head()**
- **df.shape**

  ```
  (1367, 17)
  ```

- **df.info()**

  ```
  <class 'pandas.core.frame.DataFrame'>
  RangeIndex: 1367 entries, 0 to 1366
  Data columns (total 17 columns):
   #   Column                Non-Null Count  Dtype  
  ---  ------                --------------  -----  
   0   text_content          1367 non-null   object 
   1   content_type          1367 non-null   object 
   2   word_count            1367 non-null   int64  
   3   character_count       1367 non-null   int64  
   4   sentence_count        1367 non-null   int64  
   5   lexical_diversity     1367 non-null   float64
   6   avg_sentence_length   1367 non-null   float64
   7   avg_word_length       1367 non-null   float64
   8   punctuation_ratio     1367 non-null   float64
   9   flesch_reading_ease   1288 non-null   float64
   10  gunning_fog_index     1332 non-null   float64
   11  grammar_errors        1367 non-null   int64  
   12  passive_voice_ratio   1336 non-null   float64
   13  predictability_score  1367 non-null   float64
   14  burstiness            1367 non-null   float64
   15  sentiment_score       1313 non-null   float64
   16  label                 1367 non-null   int64  
  dtypes: float64(10), int64(5), object(2)
  memory usage: 181.7+ KB
  ```

- `df.describe().T`

	count	mean	std	min	25%	50%	75%	max
word_count	1367.0	140.190929	97.410218	3.0000	61.500000	131.00000	193.00000	443.0000
character_count	1367.0	940.329188	654.335255	14.0000	410.500000	882.00000	1294.50000	2966.0000
sentence_count	1367.0	25.610095	17.867480	1.0000	11.000000	24.00000	35.00000	83.0000
lexical_diversity	1367.0	0.967646	0.026254	0.8750	0.951550	0.96920	0.98910	1.0000
avg_sentence_length	1367.0	5.486423	0.447202	3.0000	5.270000	5.48000	5.70000	8.0000
avg_word_length	1367.0	5.717783	0.279636	4.0000	5.590000	5.71000	5.83000	8.3300
punctuation_ratio	1367.0	0.027440	0.002801	0.0194	0.026100	0.02720	0.02840	0.0714
flesch_reading_ease	1288.0	52.183377	10.466570	-50.0100	47.712500	52.19000	57.32250	98.8700
gunning_fog_index	1332.0	7.556877	1.866676	1.2000	6.620000	7.51500	8.39000	27.8700
grammar_errors	1367.0	1.537674	1.912012	0.0000	0.000000	1.00000	3.00000	10.0000
passive_voice_ratio	1336.0	0.150198	0.056738	0.0500	0.099675	0.15135	0.20015	0.2500
predictability_score	1367.0	62.779049	28.223550	20.0300	39.015000	56.82000	86.64500	119.9300
burstiness	1367.0	0.427041	0.199249	0.1011	0.250000	0.40850	0.59430	0.7995
sentiment_score	1313.0	-0.007997	0.588354	-0.9993	-0.525800	-0.00620	0.50280	0.9959
label	1367.0	0.499634	0.500183	0.0000	0.000000	0.00000	1.00000	1.0000

Description:

In this step, essential Python libraries such as **NumPy, Pandas, Matplotlib**, and **Seaborn** are imported to support data processing and visualization. The dataset is then uploaded into the Google Colab environment using the Pandas **read_csv** function. Basic exploration is performed using functions like **head(), shape, and info()** to understand the dataset structure, size, data types, and presence of missing values. Summary statistics are obtained using **describe()** to gain an initial understanding of numerical feature distributions.

3.2 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is performed to understand data distribution, class balance, and relationships between features and the target variable. Visualizations are used to identify patterns that may influence model performance. Following analysis are performed under this portion:

Analysis	**Description**
1. *Content type distribution by Label*	This analysis examines how different content types (e.g., blogs, essays, academic content) are distributed across the target classes (AI-generated vs human-written).
2. *Target Variable Distribution*	This analysis visualizes the distribution of the target variable (labels) in the dataset.
3. *Univariate Analysis*	Univariate analysis focuses on examining the distribution of individual features independently.
4. *Bivariate Analysis (Feature vs Target)*	Bivariate analysis explores the relationship between each feature and the target variable.
5. *Multivariate Analysis (Feature Relationships)*	Multivariate analysis examines relationships among multiple features simultaneously, often using correlation analysis.

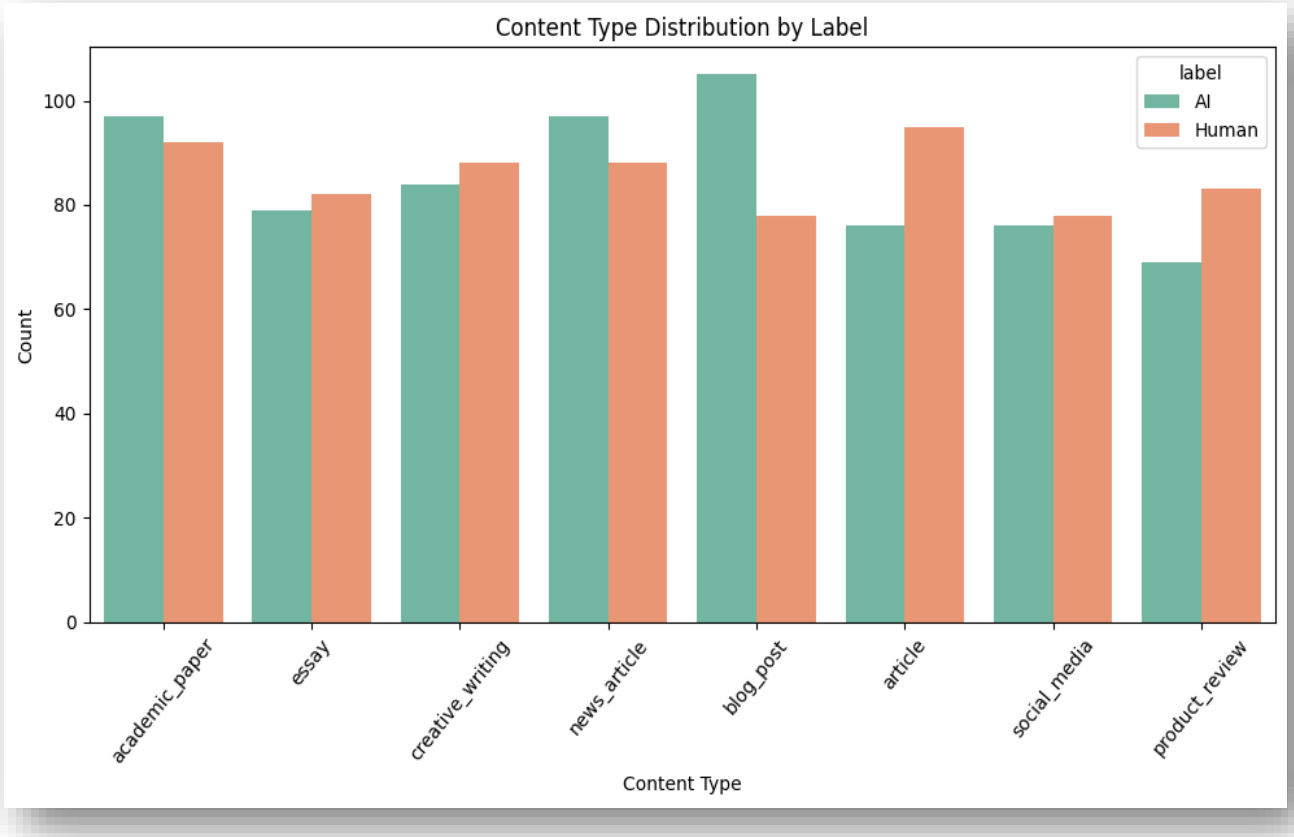
3.2.1 Content type distribution by Label

Code

```
# Check if 'label' is numeric and convert for visualization
df['label'] = df['label'].map({0: 'Human', 1: 'AI'})

# Plot barplot of content_type with hue by label
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='content_type', hue='label', palette='Set2')
plt.title("Content Type Distribution by Label")
plt.xlabel("Content Type")
plt.ylabel("Count")
plt.xticks(rotation=50)
plt.tight_layout()
plt.show()
```

Diagram

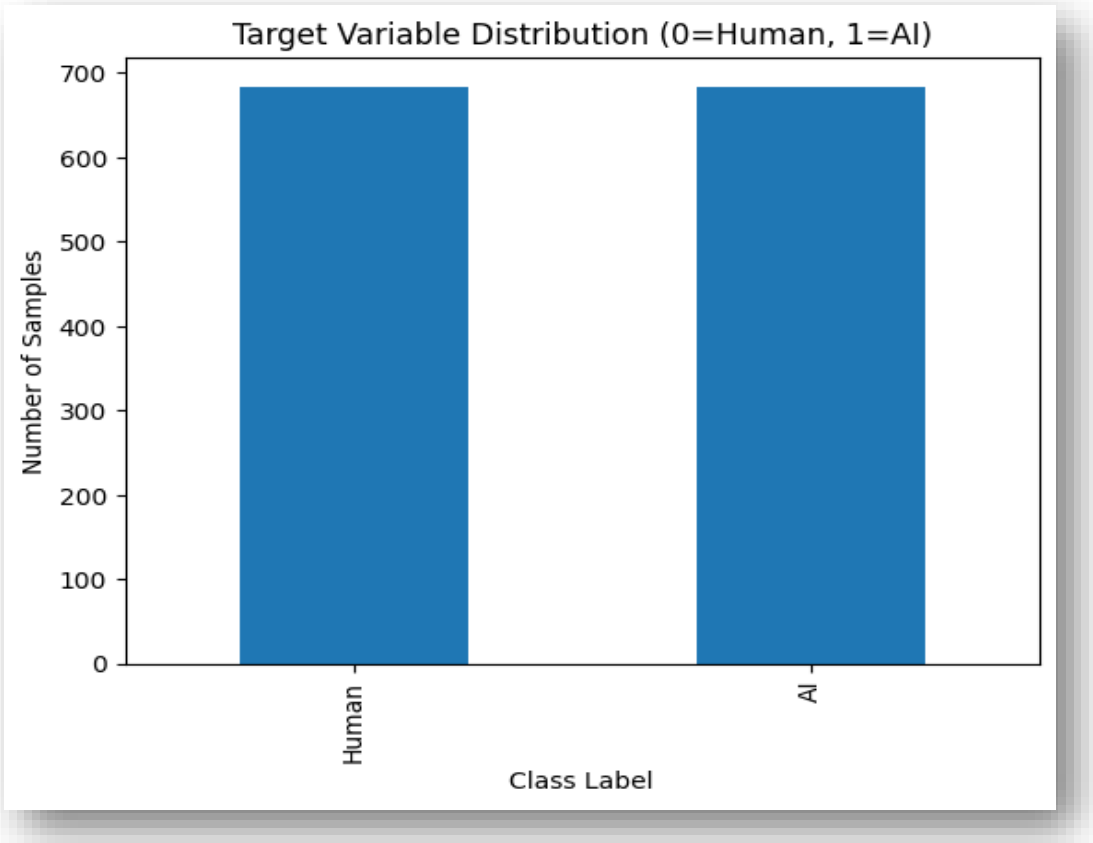


3.2.2 Target Variable Distribution

Code

```
# Count the number of samples for each class
label_counts = df['label'].value_counts()
# Display counts
print(label_counts)
plt.figure()
label_counts.plot(kind='bar')
plt.xlabel('Class Label')
plt.ylabel('Number of Samples')
plt.title('Target Variable Distribution (0=Human, 1=AI)')
plt.show()
```

Diagram



Description

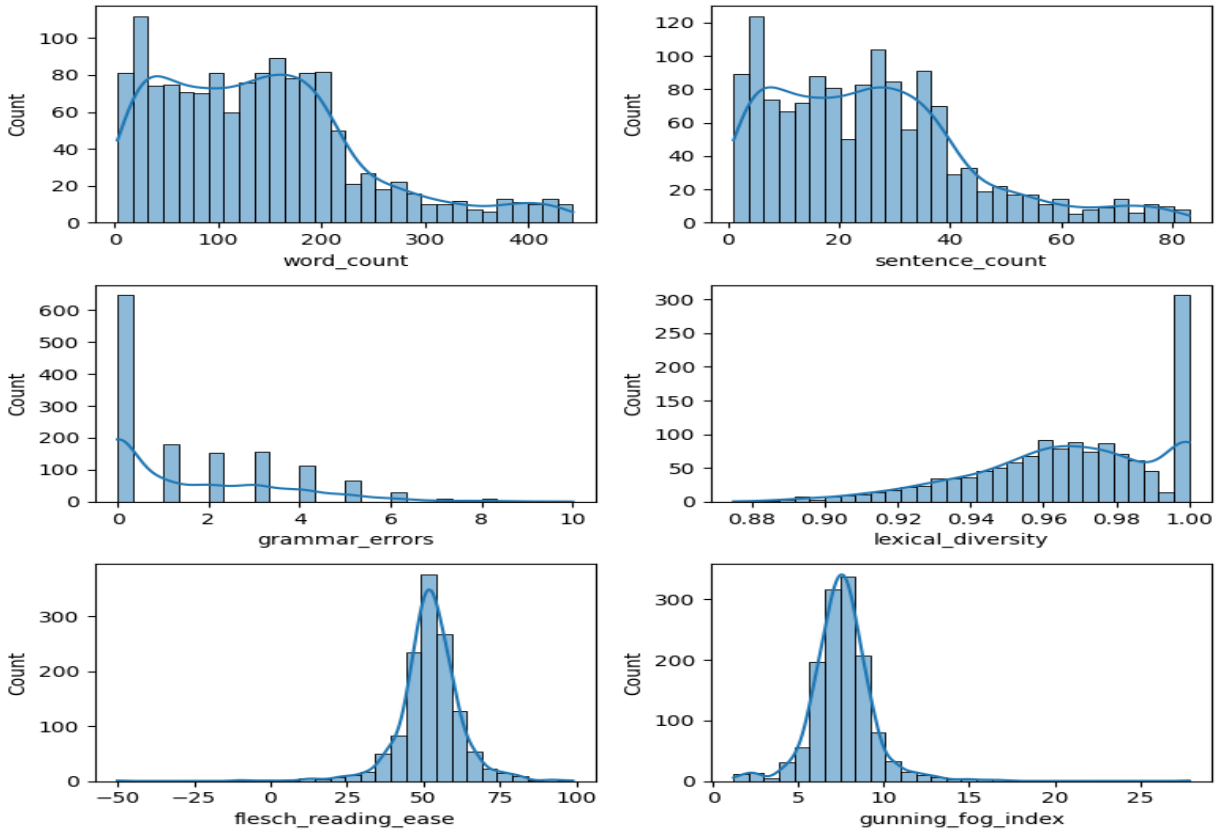
The target variable distribution shows that the dataset is **well balanced** between human-written (0) and AI-generated (1) content, with a nearly equal number of samples in both classes. This balance is important as it prevents model bias toward a dominant class during training. A balanced target distribution allows accuracy and other evaluation metrics to be reliable and meaningful. Overall, the dataset is suitable for fair and effective binary classification.

3.2.3 Univariate Analysis

Code

```
plt.figure(figsize=(8, 8))
def plotting(var,num):
    plt.subplot(3,2,num)
    sns.histplot(df[var], bins=30, kde = True)
plotting("word_count",1)
plotting("sentence_count",2)
plotting("grammar_errors",3)
plotting("lexical_diversity",4)
plotting("flesch_reading_ease",5)
plotting("gunning_fog_index", 6)
plt.tight_layout()
```

Diagram



Description

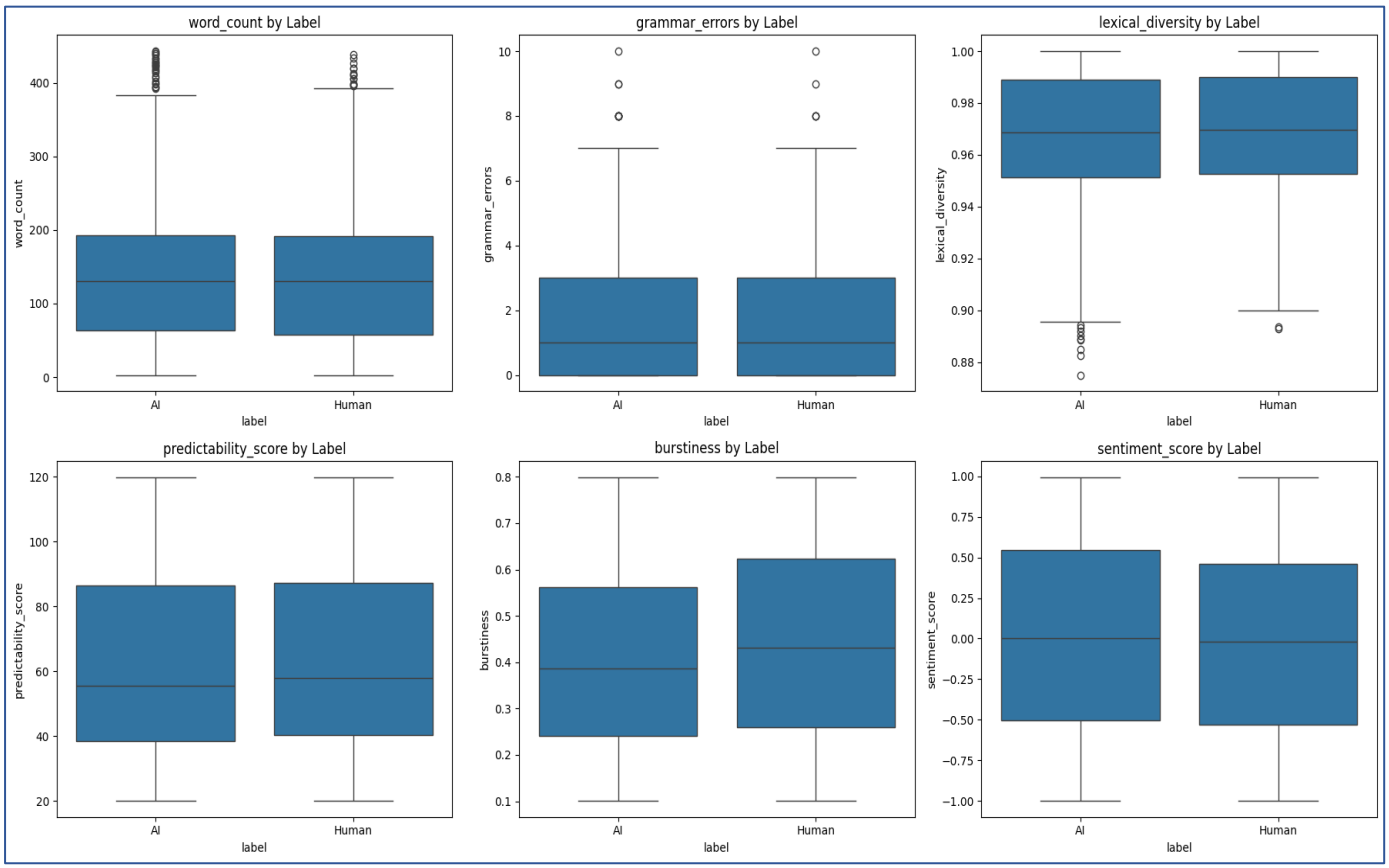
- **The word count and sentence count** features exhibit right-skewed distributions, indicating that most text samples are relatively short, with a smaller number of longer documents.
- **Grammar errors** are mostly concentrated at lower values, suggesting that the majority of content is grammatically sound.
- **Lexical diversity** is highly concentrated toward higher values, showing limited variation in vocabulary richness across samples.
- **Readability metrics** such as Flesch Reading Ease and Gunning Fog Index display clear distribution patterns, reflecting differences in text complexity and readability that may help distinguish AI-generated and human-written content.

3.2.4 Bivariate Analysis (Feature vs Target)

Code

```
features_to_plot = ['word_count', 'grammar_errors', 'lexical_diversity',
'predictability_score', 'burstiness', 'sentiment_score']
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10))
axes = axes.flatten()
for i, col in enumerate(features_to_plot):
    sns.boxplot(data=df, x='label', y=col, ax=axes[i])
    axes[i].set_title(f"{col} by Label")
plt.tight_layout()
plt.show()
```

Diagram



Description

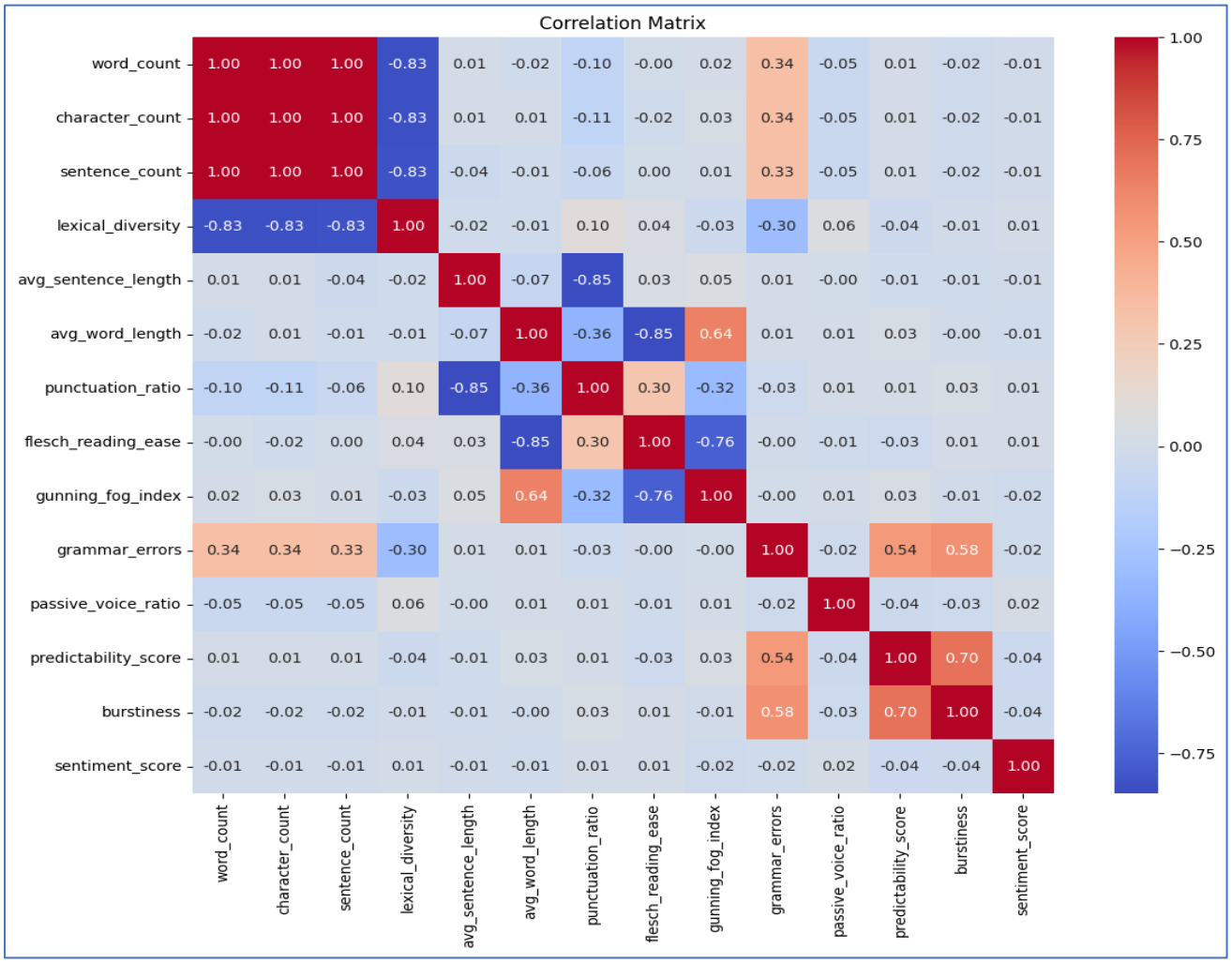
- **The word count** distribution shows a comparable range for both AI-generated and human-written content, indicating that text length alone is not a strong distinguishing factor.
- **Grammar errors** are generally low for both classes, suggesting that both AI and human texts are grammatically well-formed, though slight variation exists.
- **Lexical diversity** values are high for both classes, with human-written content showing marginally greater variability, reflecting more diverse vocabulary usage.
- **Predictability score** and **burstiness** show noticeable differences between AI and human text, indicating their potential usefulness in distinguishing writing patterns.
- **Sentiment scores** appear broadly distributed across both classes, suggesting that sentiment alone may have limited discriminative power.

3.2.5 Multivariate Analysis (Feature Relationships)

Code

```
plt.figure(figsize=(14, 10))
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", square=True)
plt.title("Correlation Matrix")
plt.show()
```

Diagram



Description

- It shows the correlation matrix of extracted features. **Word count, character count, and sentence count** are highly correlated, indicating redundancy among length-based attributes. **Lexical diversity** is strongly negatively correlated with text length, reflecting reduced vocabulary variation in longer texts.
- Readability measures exhibit strong interdependence, with **Flesch Reading Ease** inversely correlated with the **Gunning Fog Index**. **Average word length** is positively associated with text complexity. Stylistic features, including **grammar errors, predictability score, and burstiness**, show moderate positive correlations, suggesting their relevance for AI–human text discrimination. **Sentiment score** and **passive voice ratio** display weak correlations, indicating limited discriminative contribution.

3.3 Data Cleaning & Preprocessing

3.3.1 Check Missing Values

```
df.isnull().sum()
```

```
text_content              0
content_type              0
word_count                0
character_count           0
sentence_count            0
lexical_diversity         0
avg_sentence_length       0
avg_word_length           0
punctuation_ratio         0
flesch_reading_ease      79
gunning_fog_index        35
grammar_errors            0
passive_voice_ratio      31
predictability_score      0
burstiness                0
sentiment_score          54
label                     0
dtype: int64
```

```
plt.figure(figsize=(12, 8))
sns.heatmap(df.isnull(), cbar=False, cmap='Reds')
plt.title("Missing Values Heatmap")
plt.show()
```

3.3.2 Handling Missing Values

Code

```
# List of columns with missing values
missing_cols = [
    "flesch_reading_ease",
    "gunning_fog_index",
    "passive_voice_ratio",
    "sentiment_score"
]

# Fill missing values using median
for col in missing_cols:
    median_value = df[col].median()
    df[col] = df[col].fillna(median_value)

# Verify that missing values are handled
df.isnull().sum()
```

Output

```
text_content            0
content_type            0
word_count              0
character_count         0
sentence_count          0
lexical_diversity       0
avg_sentence_length     0
avg_word_length         0
punctuation_ratio       0
flesch_reading_ease     0
gunning_fog_index       0
grammar_errors          0
passive_voice_ratio     0
predictability_score    0
burstiness              0
sentiment_score         0
label                   0
dtype: int64
```

Description

Missing values were observed in readability and linguistic features such as Flesch Reading Ease, Gunning Fog Index, Passive Voice Ratio, and Sentiment Score. Since these are continuous numerical variables and the proportion of missing values was small, median imputation was applied to preserve dataset size and reduce the impact of outliers.

3.3.3 Checking Duplicate Values

Code

```
# Count duplicate rows
duplicate_count = df.duplicated().sum()
duplicate_count
```

Output

```
np.int64(0)
```

Description

No duplicate records were found in the dataset, indicating that each instance represents a unique content sample.

3.3.4 Remove Unnecessary Columns

Code

```
df.columns
```

Output

```
Index(['text_content', 'content_type', 'word_count', 'character_count',
       'sentence_count', 'lexical_diversity', 'avg_sentence_length',
       'avg_word_length', 'punctuation_ratio', 'flesch_reading_ease',
       'gunning_fog_index', 'grammar_errors', 'passive_voice_ratio',
       'predictability_score', 'burstiness', 'sentiment_score', 'label'],
      dtype='object')
```

Code

```
# Drop raw text column (not used in numerical ML models)
df.drop(columns=["text_content"], inplace=True)
df.columns
```

Output

```
Index(['content_type', 'word_count', 'character_count', 'sentence_count',
       'lexical_diversity', 'avg_sentence_length', 'avg_word_length',
       'punctuation_ratio', 'flesch_reading_ease', 'gunning_fog_index',
       'grammar_errors', 'passive_voice_ratio', 'predictability_score',
       'burstiness', 'sentiment_score', 'label'],
      dtype='object')
```

Description

The raw text feature (text_content) was removed since the modeling pipeline relies on numerical and categorical features only, and no text vectorization techniques were applied. All remaining features represent engineered linguistic, readability, and stylistic metrics relevant for AI–human content classification.

3.3.5 Encode Categorical Features (using Onehot Encoding)

Code

```
# Converting lable feature to numeric
df['label'] = df['label'].map({'Human': 0, 'AI': 1})

# Identify categorical columns
df.select_dtypes(include=["object"]).columns

Index(['content_type'], dtype='object')

# One-Hot Encode the categorical feature
df = pd.get_dummies(df, columns=["content_type"], drop_first=True)

df.head()

{"type":"dataframe","variable_name":"df"}

# Convert boolean columns to integers
bool_cols = df.select_dtypes(include="bool").columns
df[bool_cols] = df[bool_cols].astype(int)
```

Description

The categorical feature content_type was converted into numerical form using one-hot encoding. This approach prevents the introduction of artificial ordinal relationships and ensures compatibility with machine learning algorithms such as Logistic Regression, SVM, and Random Forest.

3.4 Feature Engineering

3.4.1 Separate Features and Target Variables

Code

```
# Features (all columns except target)
X = df.drop(columns=["label"])
# Target variable
y = df["label"]
# Verify shapes
X.shape, y.shape
```

Output

```
((1367, 21), (1367,))
```

3.4.2 Train Test Split

Code

```
from sklearn.model_selection import train_test_split

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Verify shapes
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Output

```
((1093, 21), (274, 21), (1093,), (274,))
```

3.4.3 Feature Scaling (using Standard Scaler)

Code

```
# Identify one-hot encoded columns
binary_cols = [col for col in X_train.columns if
col.startswith("content_type_")]

# Identify numerical columns
numerical_cols = [col for col in X_train.columns if col not in binary_cols]
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Scale numerical features
X_train_scaled_num = scaler.fit_transform(X_train[numerical_cols])
X_test_scaled_num = scaler.transform(X_test[numerical_cols])

# Convert back to DataFrame
X_train_scaled_num = pd.DataFrame(X_train_scaled_num,
columns=numerical_cols)
X_test_scaled_num = pd.DataFrame(X_test_scaled_num,
columns=numerical_cols)

# Add binary features back without scaling
X_train_final = pd.concat([X_train_scaled_num,
X_train[binary_cols].reset_index(drop=True)], axis=1)
X_test_final = pd.concat([X_test_scaled_num,
X_test[binary_cols].reset_index(drop=True)], axis=1)
```

Description

Continuous numerical features were standardized using StandardScaler. One-hot encoded categorical features were excluded from scaling to preserve their binary semantics and improve model interpretability.

3.5 Model Building

Naïve Bayes, Logistic Regression, Gradient Boost, SVM, KNN, XGBoost are used.

Code

```
# Import models
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Try importing XGBoost (optional / advanced)
try:
    from xgboost import XGBClassifier
    xgboost_available = True
except ImportError:
    xgboost_available = False

# Dictionary of models
models = {
    "Naive Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42, n_jobs=-1
    ),
    "SVM": SVC(kernel="linear", random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=9),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

# Add XGBoost if available
if xgboost_available:
    models["XGBoost"] = XGBClassifier(
        eval_metric="logloss",
        random_state=42
    )

# Train all models
trained_models = {}

for model_name, model in models.items():
    model.fit(X_train_final, y_train)
    trained_models[model_name] = model
    print(f"{model_name} trained successfully.")
```

Output

```
Naive Bayes trained successfully.
Logistic Regression trained successfully.
Random Forest trained successfully.
SVM trained successfully.
KNN trained successfully.
Gradient Boosting trained successfully.
XGBoost trained successfully.
```

3.6 Model Evaluation

Following classification evaluation metrics are used:

	Metric	**Description**
1.	*Accuracy*	Measures how many predictions the model got correct out of all predictions.
2.	*Precision*	Precision quantifies the proportion of correctly predicted positive instances among all predicted positives.
3.	*Recall*	Recall (also known as sensitivity) measures the proportion of actual positive instances that are correctly identified by the model.
4.	*F1-Score*	The F1-score is the harmonic mean of precision and recall, providing a balanced measure when class distributions are uneven.

3.6.1 Generate Predictions & Metrics

Code

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score
results = []
for model_name, model in trained_models.items():
    y_pred = model.predict(X_test_final)

    results.append({
        "Model": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred,
average='weighted'),
        "Recall": recall_score(y_test, y_pred, average='weighted'),
        "F1 Score": f1_score(y_test, y_pred, average='weighted')
    })

results_df = pd.DataFrame(results)
results_df
```

Output

Model	*Accuracy*	*Precision*	*Recall*	*F1 Score*
Naive Bayes	0.532847	0.532910	0.532847	0.532623
Logistic Regression	0.503650	0.503653	0.503650	0.503544
Random Forest	0.492701	0.492489	0.492701	0.489101
SVM	0.474453	0.473776	0.474453	0.471042
KNN	0.543796	0.544481	0.543796	0.542033
Gradient Boosting	0.496350	0.496347	0.496350	0.496243
XGBoost	0.503650	0.503678	0.503650	0.502696

3.7 Model Evaluation

Following classification evaluation metrics are used:

	Metric	**Description**
2.	*Accuracy*	Measures how many predictions the model got correct out of all predictions.
5.	*Precision*	Precision quantifies the proportion of correctly predicted positive instances among all predicted positives.
6.	*Recall*	Recall (also known as sensitivity) measures the proportion of actual positive instances that are correctly identified by the model.
7.	*F1-Score*	The F1-score is the harmonic mean of precision and recall, providing a balanced measure when class distributions are uneven.

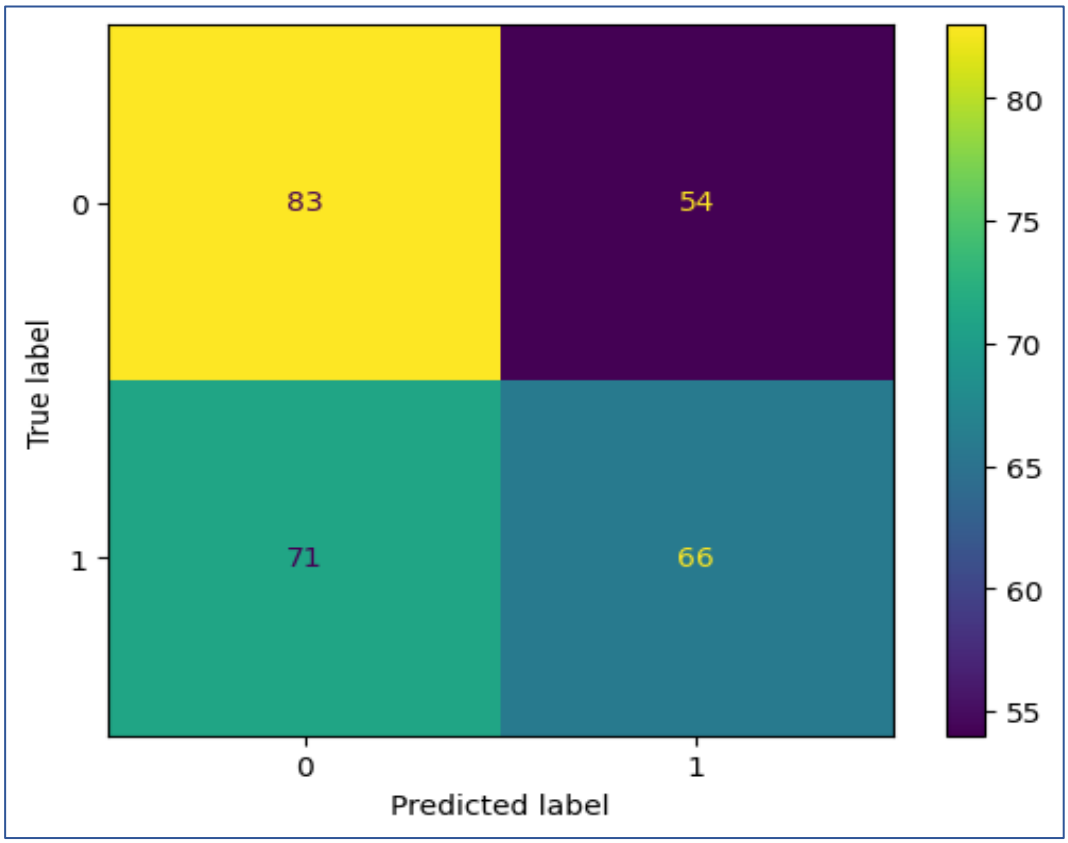
3.7.1 Display Confusion Matrix (for best Accuracy Model)

Code

```
best_model_name = results_df.sort_values(by="Accuracy",
ascending=False).iloc[0]["Model"]
best_model = trained_models[best_model_name]
print("Best Model:", best_model_name)
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
y_pred_best = best_model.predict(X_test_final)
cm = confusion_matrix(y_test, y_pred_best)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

Output

```
Best Model: KNN
```



Description

The confusion matrix illustrates the model's ability to distinguish between AI-generated and human-written text. The model correctly identifies **83 AI-generated texts** and **66 human-written texts**, while **54 AI-generated samples are incorrectly classified as human-written** and **71 human-written samples are misclassified as AI-generated**. These misclassifications indicate overlap in writing patterns between AI and human text, highlighting the inherent challenge of accurately separating the two classes.

Chapter 4

Conclusion

4. Conclusion

4.1 Conclusion

This study investigated the problem of distinguishing AI-generated text from human-written content using a structured data science approach and supervised machine learning techniques. A complete analytical pipeline was implemented, encompassing exploratory data analysis, data preprocessing, feature-based representation, model training, and performance evaluation. Several classification models were developed and compared using standard evaluation metrics. The experimental results indicate that although the overall classification accuracy remains moderate, the selected linguistic and statistical features provide meaningful discriminatory information between AI-generated and human-authored text. Nevertheless, this work establishes a solid baseline and contributes valuable insights for further research in automated AI content detection.

4.2 What I Learned?

- Developed the ability to design and implement a complete data science workflow, from data ingestion to model evaluation.
- Strengthened skills in exploratory data analysis to extract insights and understand feature behavior.
- Gained practical experience in applying and comparing multiple machine learning classification algorithms.
- Learned to evaluate model performance using appropriate metrics and interpret moderate results in the context of real-world data complexity.
- Enhanced understanding of feature importance analysis and its role in model interpretability.
- Improved critical thinking and problem-solving skills by identifying limitations and potential areas for improvement in machine learning solutions.